

SPEC 2111

Deep Learning and Neural Networks

Öznur Alkan

Staff Technical Lead @ HubSpot, AI Infra Group

oznurkirmemis@gmail.com

Education

- PhD, MS, and BSc in Computer Engineering – METU, Türkiye
 - PhD focus: Data Mining
 - MS focus: Artificial Intelligence

Experience

- A Mix of Research, Academia, and Industry
 - Completed PhD, MS studies in parallel with industry experience.
- Previous Roles
 - Senior Principal Engineer – AI, Optum / UnitedHealth Group
 - Research Scientist & Master Inventor – AI, IBM Research
 - Assistant Professor – Computer Engineering, Yıldırım Beyazıt University
 - Senior Software Engineer / Software Engineer – R&D, Siemens
 - Authored / Co-authored 25+ patents, 30+ publications

Outside Work

- Mum to twins
- Long-distance running
- Audiobooks (fiction & mystery)

Week-1 / Agenda

Theme: Course Overview and a Gentle Intro

- Prerequisites
- Course Structure
- Coding Environment
- Assessment
- Course Scope & Schedule
- A Gentle Intro to Deep Learning
- Lab 1 - Set up



Prerequisites

- Familiarity with fundamental ML concepts
 - Train/test datasets, types of learning, overfitting vs. underfitting, etc.
 - ML concepts most relevant to Neural Networks and Deep Learning will be covered in Weeks 2–3.
- Familiarity with Python
 - All lab assignments will be coded in Python.
 - Refresh basic Python syntax (functions, loops, conditionals, etc), NumPy, Jupyter Notebooks
 - Start with Week-1 notebook

Course Structure

- Lectures are 2 hours per week
 - Wednesdays @ 9-11am
- Each week starts with a theory-based lecture covering key concepts for that topic
 - Lecture notes will be provided
- This is followed by a lab session focused on hands-on learning
 - You can explore Jupyter notebooks shared before the class
 - Lab time is used for experimentation and discussion



Coding Environment



- [Google Colab](#) for course materials and in class demonstrations
 - hosted Jupyter Notebook service that requires no setup to use
 - comes with most common ML/DL libraries preinstalled (NumPy, SciPy, scikit-learn, TensorFlow, PyTorch, matplotlib, etc.)
- Setting up your own Python environment to learn and experiment is encouraged.
 - Local environments provide valuable hands-on experience with real development setups
 - For local env check Colab python version and libraries

Assessments



- Class Tests (40%)
 - 2 in class assessments (Week 7, Week 11)
 - Open book tests
 - Focused primarily on problem solving
- Final Project (60%)
 - Final Project due at the end of the semester
 - Details will be announced

Course Scope

- Focus: Study key concepts in Neural Networks & Deep Learning
 - Emphasis on balanced theory and practice

Course Outline and Scope

- Focus: Study key concepts in Deep Learning
 - Emphasis on balanced theory and practice
- Learning Resources
 - Course is mostly self contained
- Additional Resources
 - [Deep Learning](#) with Python by Francois Chollet
 - [Hands-on](#) Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems by Aurélien Géron
 - [Collection](#) of Deep Learning Resources: curated list of tutorials, books, and courses including TensorFlow and Keras.

Course Schedule



Course Schedule

PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview* 
- *Week 2 – Linear Regression*
 - *Regression concepts, gradient descent, loss functions*
- *Week 3 – Logistic Regression*
 - *Classification, sigmoid, cross-entropy loss*

Course Schedule

PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview* 
- *Week 2 – Linear Regression*
 - *Regression concepts, gradient descent, loss functions*
- *Week 3 – Logistic Regression*
 - *Classification, sigmoid, cross-entropy loss*

PART-2: Neural Networks, Deep Learning and Training

- *Week 4 – Neural Network Essentials*
 - *Neurons, layers, forward propagation, activation functions*
- *Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)*
 - *Backpropagation, chain rule, gradient descent optimization, intro to multiple layers*
- *Week 6 – Preventing Overfitting*
 - *Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping*

Course Schedule



PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview* ←
- *Week 2 – Linear Regression*
 - Regression concepts, gradient descent, loss functions
- *Week 3 – Logistic Regression*
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- *Week 4 – Neural Network Essentials*
 - Neurons, layers, forward propagation, activation functions
- *Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)*
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- *Week 6 – Preventing Overfitting*
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- *Week 7 – Convolutional Neural Networks (CNNs)*
 - Convolutions, pooling, image applications
 - *Assessment-1*
- *Week 8 – Representations & Transfer Learning (and Test)*
 - Feature learning, pre-trained models, fine-tuning
- *Week 9 – Recurrent Neural Networks & LSTM*
 - RNNs, LSTM/GRU, sequence modeling
- *Week 10 – Attention & Transformers*
 - Attention mechanism, transformer architecture, NLP applications

Course Schedule



PART-1: Introduction, Foundations & Essentials

- **Week 1 – Introduction & Course Overview** ←
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- Week 6 – Preventing Overfitting
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - **Assessment-1**
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

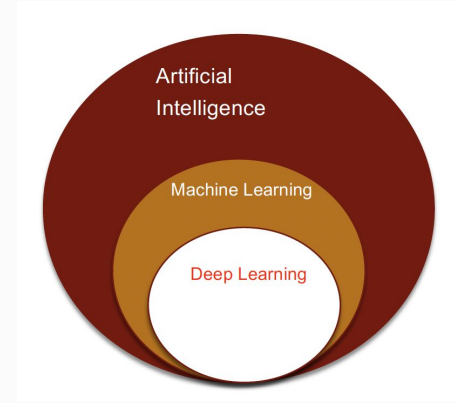
PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - **Assessment-2**
- Week 12 - Project Presentations & Discussions (**Project Due**)

How AI, ML, DL fit together

Let's start with a very quick introduction to the relationship between

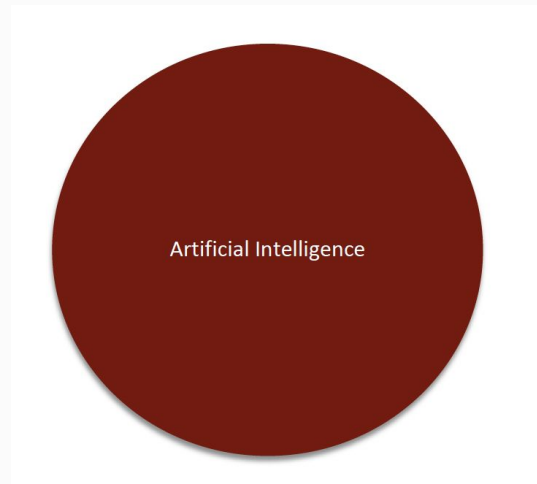
- Artificial Intelligence (AI)
- Machine Learning (ML)
- Deep Learning (DL)



We will talk about how we get to
AI-> ML->DL

Artificial Intelligence

- Originated in the 1950s
 - researches first began asking “whether machines could think”
- AI can be described as *the effort to automate intellectual tasks normally performed by humans.*
- Early AI work focused on rule-based, “traditional” AI systems
 - Ask human experts how they solve a task
 - Translate into explicit if-then rules
 - Manually program rules into the system

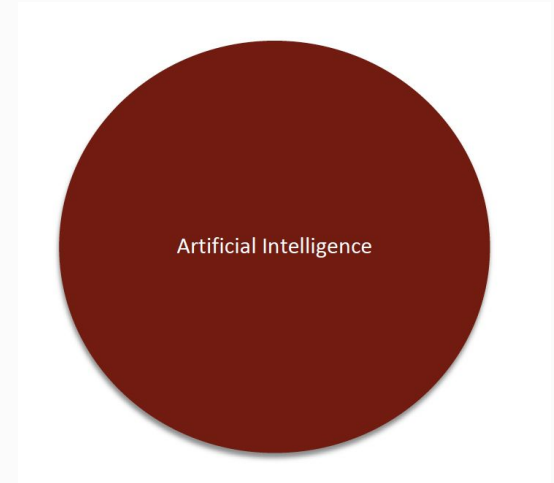


*IF a patient has symptom A AND symptom B,
THEN diagnose disease X*

*IF the chess board looks like this, THEN make
move Y*

Traditional AI (Rule-Based Systems)

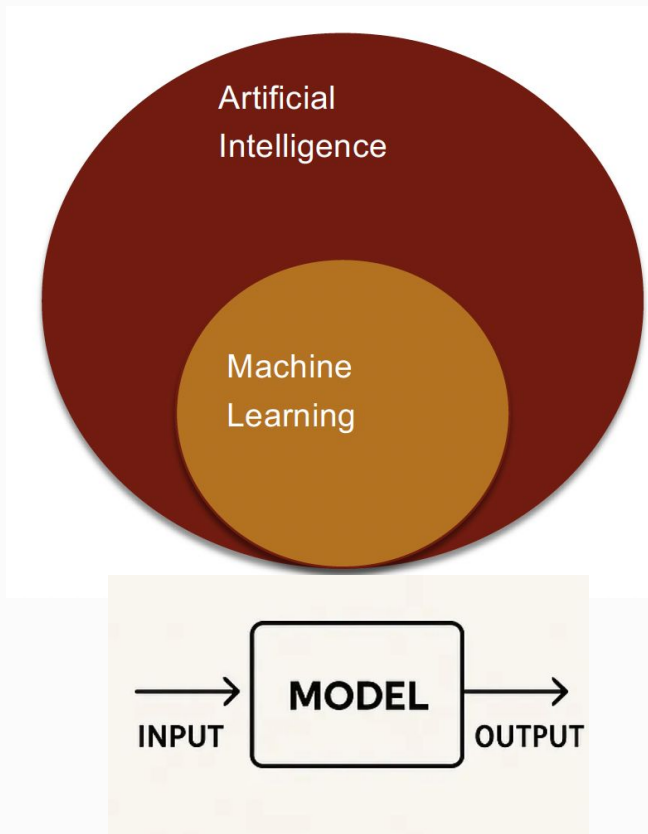
- Example:
 - *IF a patient has symptom A AND symptom B, THEN diagnose disease X*
 - *IF the chess board looks like this, THEN make move Y*
- Success is limited!!
 - *can not write down if-then rules to cover all possible situations, edge cases etc.*
- Limitations
 - required human effort
 - did not generalize well to new situations



From Rules to Learning

Machine Learning: Learning from Examples

- *Hand-coded rules don't scale*
- Instead of programming rules, learn rules from data automatically using statistical techniques to learn relationships
- Many types of ML models exist:
 - Linear Regression, Random Forests, Support Vector Machines (SVMs), Neural Networks



Learning Effective Feature Representations is Vital

- Heavy reliance on manual feature engineering
 - Large effort spent on data preparation/feature representation
 - Performance depends on hand-crafted features

Learning Effective Feature Representations is Vital

- Example: User Shopping Over a Month
 - Goal: Predict whether the user will make a purchase on the next day
 - Consider a user's shopping activity over 4 weeks: [1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0]
 - 1 = user made a purchase that day, 0 = user did not purchase
- Input to model: vector of fixed features

Learning Effective Feature Representations is Vital

- Example: User Shopping Over a Month
 - Goal: Predict whether the user will make a purchase on the next day
 - Consider a user's shopping activity over 4 weeks: [1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0]
 - 1 = user made a purchase that day, 0 = user did not purchase
- Input to model: vector of fixed features

Standard ML

- Total purchases last month = 8
- Weekday purchases = 8
- Weekend purchases = 0
- Days since last visit = 2

Learning Effective Feature Representations is Vital

- Example: User Shopping Over a Month
 - Goal: Predict whether the user will make a purchase on the next day
 - Consider a user's shopping activity over 4 weeks: [1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0]
 - 1 = user made a purchase that day, 0 = user did not purchase
- Input to model: vector of fixed features

Standard ML

- Total purchases last month = 8
- Weekday purchases = 8
- Weekend purchases = 0
- Days since last visit = 2

Limitations

- Human effort required to decide, engineer, and update features
- Temporal order and sequence patterns are lost
 - a. Ex: a purchase followed by no-purchase

Learning Effective Feature Representations is Vital

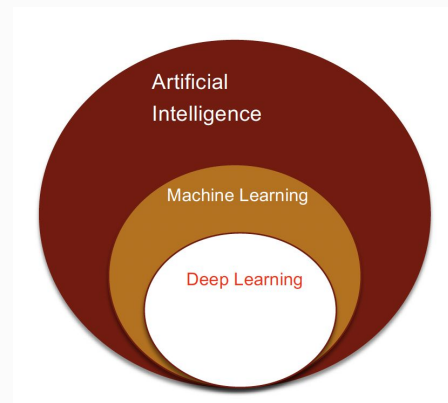
- Heavy reliance on manual feature engineering
 - Large effort spent on data preparation/feature representation
 - Performance depends on hand-crafted features
- Struggles with high dimensional unstructured data (images, text, and audio)
 - Curse of dimensionality (distances between points become less meaningful, hence traditional ML struggle to find patterns)

Learning Effective Feature Representations is Vital

- Heavy reliance on manual feature engineering
 - Large effort spent on data preparation/feature representation
 - Performance depends on hand-crafted features
- Struggles with high dimensional unstructured data (images, text, and audio)
 - Curse of dimensionality (distances between points become less meaningful, hence traditional ML struggle to find patterns)
- Feature pipelines are brittle and task-specific
 - Small changes in data can break the system
 - Re-engineering features might be needed

Learning Effective Feature Representations is Vital

- Heavy reliance on manual feature engineering
 - Large effort spent on data preparation/feature representation
 - Performance depends on hand-crafted features
- Struggles with high dimensional unstructured data (images, text, and audio)
 - Curse of dimensionality (distances between points become less meaningful, hence traditional ML struggle to find patterns)
- Feature pipelines are brittle and task-specific
 - Small changes in data can break the system
 - Re-engineering features might be needed
- Deep Learning solves the bottleneck!!



How Deep Learning Solves the Bottleneck

- Learns feature representations automatically from raw data
- Instead of manually defining and engineering features, model learns multiple layers of representation where each layer builds on previous
- Deep network can be considered as a multistage information distillation process, where information goes through successive filters and comes out increasingly purified.

Learning Effective Feature Representations is Vital

- Example: User Shopping Over a Month
 - Goal: Predict whether the user will make a purchase on the next day
 - Consider a user's shopping activity over 4 weeks: [1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0, 1,0,0,1,0,0,0]
 - 1 = user made a purchase that day, 0 = user did not purchase
- Input to model: vector of fixed features

Limitations of Standard ML

- Human effort required to decide, engineer, and update features
- Temporal order and sequence patterns are lost

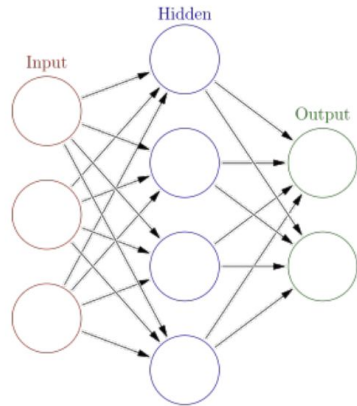
Deep Learning: Learning Patterns Automatically

- Feed the raw sequence directly into model
- DL automatically discovers patterns over time (e.g., purchases every Monday & Thursday) & complex interactions in the data
- No need for manual feature engineering
- Model adapts to new data patterns without changing input features

Why Deep Learning Works Now

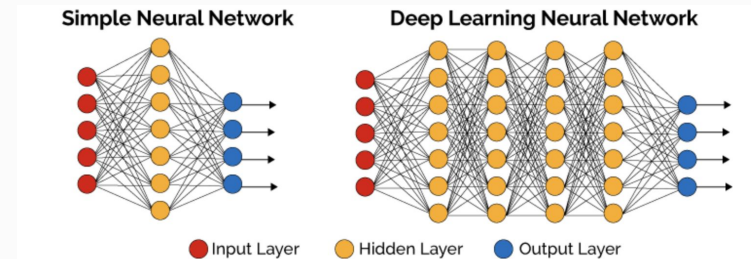
- Deep Learning = modern neural networks
- Emerged from the confluence of three forces:
 - New algorithmic ideas → improved learning techniques for neural networks
 - Lots of data → huge datasets for training
 - Parallel computing hardware → GPUs for fast computation
- These forces together revived an old concept: neural networks

New algorithmic ideas +
lots of data + computing
power applied to..



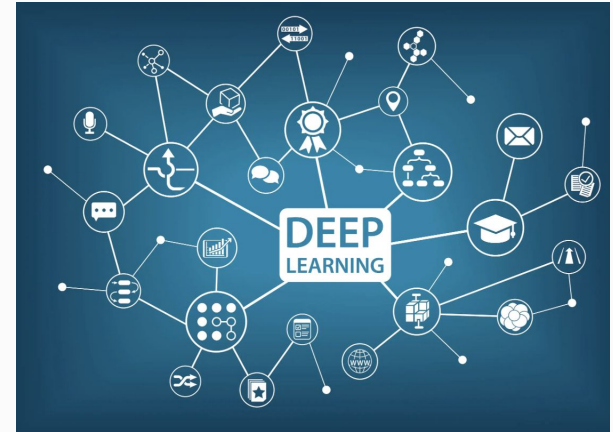
Neural Network & DL

- Neural Network
 - made of neurons organized in input → hidden → output
 - typically only one hidden layer.
- Deep Learning
 - Neural networks with multiple hidden layers - hence deep!
 - These layers allow the network to learn hierarchical features automatically



Deep Learning Today

- Deep Learning powers most AI applications we interact with today
 - ChatGBT, Smartphones: Face unlock, fingerprint/gesture recognition, Self-driving cars, Medical image analysis, drug discovery
 - Essentially, any application that perceives, understands, or generates data at scale often relies on deep learning under the hood.



Course Roadmap: From ML to Deep Learning

- From traditional ML to DL in phases
- Start with linear and logistic regression — foundational concepts for neural networks (Part-1)
- Introduce neural networks — the foundation of deep learning (Part-2)
- Build understanding for more complex models (Part-3)

PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview* ←
- *Week 2 – Linear Regression*
 - Regression concepts, gradient descent, loss functions
- *Week 3 – Logistic Regression*
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

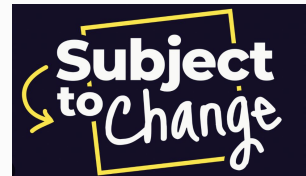
- *Week 4 – Neural Network Essentials*
 - Neurons, layers, forward propagation, activation functions
- *Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)*
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- *Week 6 – Preventing Overfitting*
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- *Week 7 – Convolutional Neural Networks (CNNs)*
 - Convolutions, pooling, image applications
- *Week 8 – Representations & Transfer Learning (and Test)*
 - Feature learning, pre-trained models, fine-tuning
- *Week 9 – Recurrent Neural Networks & LSTM*
 - RNNs, LSTM/GRU, sequence modeling
- *Week 10 – Attention & Transformers*
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- *Week 11 – Reinforcement Learning (Project Due)*
 - MDPs, Q-learning, policy gradients, neural networks in RL
- *Week 12 – Project Presentations & Discussions*



Lab for Week 1

Part 1

- Download the notebook for week 1.
- Upload it onto Google Colab and execute content
- Switch Runtime model on Google Colab between GPU and CPU.
- Can you see any difference? Probably not for such a simple model.

Part 2:

- Install Python 3.12 on your own machine
- Make sure Python is being added to your PATH
- Run python from the command line and verify the version



Appendix: DL Evolution

- Deep Learning breakthrough: ~2006 onwards
 - Large datasets became available (ImageNet, text corpora)
 - GPUs for parallel computing accelerated training
 - Better algorithms (ReLU, dropout, improved optimization)

Appendix: Gen AI Evolution

- Generative AI emerged around 2014–2017 with GANs and transformers, and became mainstream ~2022 with large-scale models.
- Early generative models:
 - Restricted Boltzmann Machines (RBMs), Variational Autoencoders (VAEs) — 2006–2014
 - GANs (Generative Adversarial Networks) introduced in 2014 (Goodfellow et al.) → big leap for generating realistic images
- Transformers: 2017 (Vaswani et al.)
 - Enabled large-scale generative models for text, images, and multi-modal data
- Modern Gen AI boom: 2022 onwards
 - ChatGPT, DALL·E, Stable Diffusion, MidJourney → widely recognized and practical generative AI